Chapter 11

Using the MySQL Security System

In previous chapters, you have been using the MySQL superuser account, `root`, to execute queries and run commands. While this is convenient, it goes contrary to one of the basic laws of multiuser system security: never use a privileged user account to perform tasks that can be performed as well with a nonprivileged account. Using a privileged account carelessly for your MySQL applications opens a security hole, and can also produce inconsistent results if your application is ever forced to run as a nonprivileged user (who has fewer capabilities and may, therefore, be unable to perform critical actions).

For this reason, it's important to understand the basics of the MySQL security subsystem, and to use it to enforce access control rules on your databases. A careful application of MySQL's privilege levels and authentication schemes can go a long way toward protecting the integrity of your data, and in ensuring that your applications work securely and consistently.

How to…

- Control access to MySQL on the basis of username and host
- Set (and reset) user passwords
- Grant and revoke user privileges to databases and tables
- Restrict the SQL commands a user is permitted to call on
- View the privileges assigned to a specific user
- Gain access to MySQL even if you lose or forget the root account password

Understanding the Need for Access Control

As you saw in previous chapters, you can only connect to the MySQL server through the MySQL client after sending the server a valid username and password. This username-password combination is used by MySQL to check which databases and tables you have access to, and which types of operations you are permitted to perform on them.

For convenience, previous chapters have directed you to use the MySQL superuser account, `root`, to execute queries and run commands. While this is acceptable for testing purposes, it cannot continue in production applications, for two reasons:

- Allowing different applications superuser access to MySQL opens a serious security hole in your RDBMS, as it makes the sensitive data of one application visible to other applications and users. This lets an application or user concerned with one database make changes to other databases on the system, and thereby interfere with the security and reliability of other applications. In the worst-case scenario, a single badly written application could destroy the databases and tables of other applications sharing the same RDBMS.
- Because all commands are routed through a single user account, it's extremely difficult to audit and log the applications of individual users or applications, because MySQL has no way of knowing which "real" person performed which command. For obvious reasons, this is a security no-no.

Given these reasons, it is preferable for every MySQL application you create to have a database of its own, to which it has complete access and that other applications cannot enter. By creating such database "bubbles" for each application, you can rest confident in the knowledge that applications are solely responsible for their own database and cannot interfere with each other. This improves security and database integrity, reduces the risk of a single application running amok, and—because each application now corresponds to a MySQL user—also makes it possible to create an audit trail for each application.

A good rule of thumb in this context is, "Grant no more permissions than are necessary for the requirements of each application." By being parsimonious with application permissions and giving each application only the minimum necessary privileges, you reduce the risk of a security breach in a single application affecting your entire MySQL installation. Spend some time thinking about exactly what actions your application needs to perform and assign privileges appropriately. The investment in time and thought will pay rich dividends in terms of a more robust and hack-proof MySQL server.

Understanding How MySQL Access Control Works

MySQL comes with a sophisticated access control and privilege system to prevent unauthorized users from accessing the system. This system, implemented as a five-tiered privilege hierarchy, enables MySQL administrators to protect access to sensitive data using a combination of user- and host-based authentication schemes. Users can be restricted to performing operations only on specified databases or fields, and MySQL even makes it possible to control which types of queries a user can run, at database, table, or field level.

All these privileges are controlled through MySQL's security subsystem which consists of five tables. These tables, known as *grant tables,* offer database administrators a great deal of power and flexibility in deciding the rules that govern access to the system. Each table has a different role to play in deciding whether a user has access to a specific database, table, or table column. Access rules may be set up on the basis of username, connecting host, or database requested.

In MySQL, access control takes place in two stages:

1. When a user requests a connection to the database server from a specific host, MySQL will first check whether there is an entry for the user in the grant tables, if the user's password is correct, and if the user is allowed to connect from that specific host. If the check is successful, a connection will be allowed to the server.
2. Once a connection is allowed, every subsequent request to the server—`SELECT`, `DELETE`, `UPDATE`, and other queries—will first be vetted to ensure that the user has the security privileges necessary to perform the corresponding action. A number of different levels of access are possible—some users may only have the ability to `SELECT` from the tables, while others may have `INSERT` and `UPDATE` capabilities, but not `DELETE` capabilities.

Normally, the MySQL superuser assigns rights and privileges to other users with the `GRANT` and `REVOKE` commands. These commands are the primary interface to the grant tables, and they should be used instead of manually modifying the tables through `INSERT` or `UPDATE` commands.

Assigning, Revoking, and Viewing User Privileges

MySQL users can be assigned any of almost 25 different privileges. Table 11-1 lists the important ones.

For a complete list of privilege levels and examples of how they may be assigned, visit the MySQL manual, at **http://dev.mysql.com/doc/mysql/en/Privileges_provided.html**.

These privilege levels can be assigned to users with the special `GRANT` command. To see how the `GRANT` command works, use the following command to assign `SELECT`, `INSERT`, `UPDATE`, and `DELETE` privileges on the table `db2.movies` to the user `joe` connecting from `localhost` with password `rosebud`:

Privilege	What It Permits
ALTER	Altering tables after they have been created
DELETE	Deleting records from tables
INSERT	Inserting records into tables
SELECT	Retrieving records from tables
UPDATE	Updating records in tables
CREATE	Creating new tables and databases
DROP	Deleting tables and databases
GRANT	Granting other users privileges
PROCESS	Viewing MySQL process information
SHUTDOWN	Shutting down the MySQL server
USAGE	Using the MySQL server

TABLE 11-1 Important MySQL Privilege Levels

```
mysql> GRANT SELECT, INSERT, UPDATE, DELETE ON db2.movies ↵
TO joe@localhost IDENTIFIED BY 'rosebud';
Query OK, 0 rows affected (1.32 sec)
```

Now, once the user logs in, only the commands specified in the `GRANT` command are permitted to the user. All other commands are denied. Every time the user requests a specific command, MySQL refers to its grant tables and only permits the command to be executed if the privilege rules allow it.

To see how this works, try logging in to the server with the username `joe` and password `rosebud`, and performing different commands:

```
$ mysql -u joe -p
Password: *******
mysql> SELECT * FROM db2.movies;
+-----+---------------------+-------+
| mid | mtitle              | myear |
+-----+---------------------+-------+
|   1 | Rear Window         |  1954 |
|   2 | To Catch A Thief    |  1955 |
|   3 | Maltese Falcon, The |  1941 |
|   4 | The Birds           |  1963 |
|   5 | North By Northwest  |  1959 |
|   6 | Casablanca          |  1942 |
|   7 | Vertigo             |  1958 |
+-----+---------------------+-------+
```

```
7 rows in set (0.11 sec)
mysql> ALTER TABLE db2.movies DROP PRIMARY KEY;
ERROR 1142: alter command denied to user: 'joe@127.0.0.1' ↵
for table 'movies'
```

Thus, only commands specified in the GRANT statement are permitted to the user.

The following command enables the user admin connecting from localhost to perform SELECT queries on the db table in the mysql database.

```
mysql> GRANT SELECT ON mysql.db TO admin@localhost IDENTIFIED BY 'secret';
Query OK, 0 rows affected (0.1 sec)
```

If you like, you can even specify which fields the user is allowed to manipulate, by naming them in the GRANT command. Here's an example:

```
mysql> GRANT SELECT (mid, mtitle) ON db2.movies ↵
TO joe@localhost IDENTIFIED BY 'rosebud';
Query OK, 0 rows affected (0.1 sec)
```

You can use the * wildcard to mean "all databases" or "all tables" in the GRANT and REVOKE commands. As an example, consider the following command:

```
mysql> GRANT SELECT ON db2.* TO guest@localhost;
Query OK, 0 rows affected (0.3 sec)
```

This lets the user guest perform SELECT operations on all tables in the db2 database.

To permit access to a user from any machine within a domain, use the % wildcard in the hostname. Consider the following command:

```
mysql> GRANT USAGE ON *.* to 'guest'@'%.goodguys.com'
Query OK, 0 rows affected (0.3 sec)
```

This enables the user guest to access the server from any system in the goodguys.com domain.

The REVOKE command does the reverse of the GRANT command, making it possible to revoke privileges assigned to a user. Consider the following example, which rescinds joe's DELETE and UPDATE rights on the db2.movies table:

```
mysql> REVOKE DELETE, UPDATE ON db2.movies FROM joe@localhost;
Query OK, 0 rows affected (0.05 sec)
```

Now, try logging in as `joe` again and performing a `DELETE`:

```
$ mysql -u joe -p
Password: *******
mysql> DELETE FROM db2.movies WHERE id = 5;
ERROR 1142: delete command denied to user: 'joe@127.0.0.1' ↵
for table 'movies'
```

MySQL also lets you assign all privileges to a user with the shortcut keyword `ALL`. This is illustrated in the next example, which grants all privileges on database `employees` to user `hr` connecting from the `company.com` domain:

```
mysql> GRANT ALL ON employees.* TO hr@'%.company.com' ↵
IDENTIFIED BY 'secret';
Query OK, 0 rows affected (0.06 sec)
```

You can also use the `ALL` keyword in a `REVOKE` command to revoke all of a user's privileges to a database, as in the following example:

```
mysql> REVOKE ALL ON employees.* FROM hr@'%.company.com';
Query OK, 0 rows affected (0.05 sec)
```

To view the privileges assigned to a user, use the `SHOW GRANTS` command with the username and hostname as argument. Here's an example:

```
mysql> SHOW GRANTS FOR joe@localhost;
+-----------------------------------------------------------+
| Grants for joe@localhost                                  |
+-----------------------------------------------------------+
| GRANT USAGE ON *.* TO 'joe'@'localhost'                   | ↵
IDENTIFIED BY PASSWORD  '2469c1e2080e09ef'                  |
| GRANT SELECT, INSERT ON db2.movies TO 'joe'@'localhost' |
+-----------------------------------------------------------+
2 rows in set (0.05 sec)
```

Working with User Accounts and Password

As you already know, a valid MySQL user account is required to connect to the MySQL server. This account may or may not be protected with a password, although using a password is recommended for the security of your MySQL installation. These user accounts and passwords are also controlled through the grant tables, and the following sections discuss how to create and use them.

Did you know?

Lockdown

MySQL 4.1 uses a new, encrypted, and highly secure protocol to handle client-server communication, which makes it harder for hackers to break your password.

Creating and Removing User Accounts

Normally, whenever you issue a `GRANT` command, MySQL automatically creates an account for the user in its grant tables (if one does not already exist, that is). If the `GRANT` command includes an `IDENTIFIED BY` clause, MySQL also encrypts the user password in that clause and stores it in the grant tables. If the `IDENTIFIED BY` clause is not present, MySQL assigns the account an empty password. Here is an example:

```
mysql> GRANT USAGE ON *.* to john@localhost;
Query OK, 0 rows affected (0.06 sec)
```

However, when a user's privileges are stripped with the `REVOKE` command, MySQL does not automatically delete the user account. To explicitly remove a user account, it is necessary to manually modify the grant tables with the `DELETE` command. For example, to remove `joe`'s user account, you would need to run the following commands:

```
mysql> DELETE FROM mysql.user WHERE User = 'john' ↵
AND Host = 'localhost';
Query OK, 1 row affected (0.06 sec)
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.28 sec)
```

If you're using MySQL 4.1.1, you can also remove a user with the new `DROP USER` command. The next command is equivalent to the previous two:

```
mysql> DROP USER john@localhost;
Query OK, 1 row affected (0.06 sec)
```

Did you know?

Starting from Zero

The `FLUSH PRIVILEGES` command forces MySQL to reread the privilege tables.

Altering User Passwords

Users with write access to the MySQL grant tables, such as `root`, can alter the passwords of other users by using the `SET PASSWORD` command. The following example sets the password for user `joe` to `guessme`:

```
mysql> SET PASSWORD FOR joe@localhost = PASSWORD('guessme');
Query OK, 1 row affected (0.06 sec)
```

Passwords can also be set in the `IDENTIFIED BY` clause of a `GRANT` command. Here is an example, which creates an account for user `guest` connecting from `localhost` with account password `guest`:

```
mysql> GRANT USAGE ON *.* to guest@localhost IDENTIFIED BY 'guest';
Query OK, 0 rows affected (0.06 sec)
```


Did you know?

Manual Labor

There is a difference between setting a password in the `IDENTIFIED BY` clause of the `GRANT` command and with the `SET PASSWORD` command. In the former case, the password is automatically encrypted by MySQL. In the latter, it must be manually encrypted with the built-in MySQL `PASSWORD()` function.

When a user logs in to the MySQL server and provides a password string, MySQL first encrypts the provided password string using the `PASSWORD()` function, and then compares the resulting value with the stored value. If the two values match (and other access rules permit it), the user is granted access. If the values do not match, access is denied.

How to ... Reset the MySQL root Password

If you forget the MySQL root password and are locked out of the system, don't panic—there's still hope! All you need to do is follow the simple steps outlined in the following:

1. Shut down MySQL, and then restart it, using the procedures outlined in Chapter 2 of this book. When restarting MySQL, add the following two options to the `mysqld_safe` command line:

```
$ /usr/local/mysql/bin/mysqld_safe --skip-grant-tables ↵
--skip-networking &
```

2. Log in to the MySQL server as `root` using the MySQL client. Because MySQL was started without the grant tables, you can use an empty password to gain access.

```
$ /usr/local/bin/mysql -u root
```

3. Once logged in, set a new password for the `root` account, by directly modifying the grant tables as follows:

```
mysql> UPDATE mysql.user SET ↵
Password=PASSWORD('new-password') WHERE User='root';
Query OK, 1 row affected (0.06 sec)
```

4. Log out of MySQL, and then shut down and restart the server in the normal manner. You should now be able to log in as the `root` user, with the new password you set in step 3.

Summary

This chapter focused on MySQL's implementation of the Data Control Language (DCL) component of SQL, teaching you how (and why) to use MySQL's security system to prevent unauthorized access to the server. The examples and commands in this chapter showed you how to create user accounts, set passwords, assign privileges, and create access rules to clearly define what each user can—and cannot—do after logging in to the MySQL server. The chapter also provided a solution to a common MySQL problem: gaining access to the server after forgetting the superuser password.

MySQL's security system is powerful and flexible, but it's only as good as the administrators who implement it. To improve your understanding of the MySQL security system and to learn more about how to secure your system against attacks, consider visiting the following links:

- A detailed discussion of the MySQL grant tables, at **http://www.melonfire.com/community/columns/trog/article.php?id=62**
- Information on MySQL privilege levels, at **http://dev.mysql.com/doc/mysql/en/Privilege_system.html**
- Information on securing MySQL against network attacks, at **http://dev.mysql.com/doc/mysql/en/Security_against_attack.html**
- General MySQL security guidelines, at **http://dev.mysql.com/doc/mysql/en/Security_guidelines.html**
- Using encrypted server connections with SSL, at **http://dev.mysql.com/doc/mysql/en/Secure_connections.html**
- Tips on password security, at **http://dev.mysql.com/doc/mysql/en/Password_security.html**